Java Encapsulation

(প্রতি **condition** এর **else** হিসেবে একটা ম্যাসেজ প্রিন্ট করে দেওয়া উচিত। আমি দিতেছি না সব)

Problem 1

Create a class Employee with private name, age and salary. Setters must reject an empty name, an age outside 18-65, and a negative salary. Provide getters and test with valid and invalid values.

```
public class Employee {
	private String name;
	private int age, salary;
	public void setName(String name) {
		if(name!=null && !name.isEmpty())
		{
			this.name = name;
		}
	}
	public void setAge(int age) {
		if(age>=18 && age<=65) {
			this.age = age;
		}
	}
	public void setSalary(int salary) {
		if (salary>=0) this.salary = salary;
	}


	public String getName() {
		return this.name;
	}
	public int getAge() {
		return this.age;
	}
	public int getSalary() {
		return this.salary;
	}
	public Employee(String name, int age, int salary){
		this.setName(name);
		this.setAge(age);
		this.setSalary(salary);
	}

}
```

name.isEmpty() এ চেক করা হয় String এর সাইজ ০ কিনা।

Problem 2

Create a class ATMCard with:

- cardNumber: read-only. Set only through the constructor, getter only (no setter) • pin: write-only. Provide setPin(int) (must be a 4-digit number, 1000 to 9999) and checkPin(int) returning boolean, but no getPin()

```
public class ATMCard {
	private int cardNumber;
	private int pin;
	public ATMCard(int cardNumber){
		this.cardNumber = cardNumber;
	}
	public int getCardNumber() {
		return this.cardNumber;
	}

	public void setPin(int pin){
		if(pin>=1000&&pin<=9999) this.pin = pin;
	}
	public boolean checkPin(int pin) {
		if(this.pin == pin) return true;
		else return false;
	}
}
```

checkPin function এ just (return this.pin==pin;) লিখলেও same কাজ হয়

Problem 3

Create a class Wallet with a private balance. Do not provide setBalance(). Provide addMoney(double), spend(double) and getBalance(). Reject negative amounts and overspending.

```
public class Wallet {
	private double balance;
	public void addmoney(double money) {
		if(money>=0) this.balance = this.balance + money;
	}
	private int overAmount = 100000;//১০০০০০ কে overAmount হিসেবে variable বানাইতেছি। variable না বানালেও করা যায়। আর amount নিজের ইচ্ছামতো।
	public void spend(double money) {
		if(money>=0 && money<=overAmount) this.balance = this.balance - money;
	}
	public double getBalance(){
		return this.balance;
	}
}
```

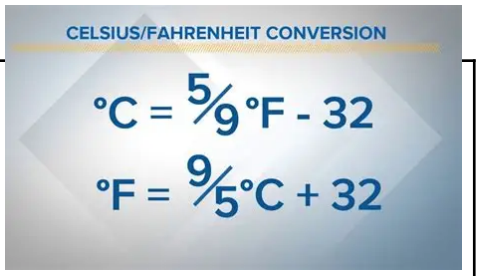
Problem 4

Create a class LibraryBook with a private title and a private boolean issued. Do not provide setIssued(). Provide getTitle(), isIssued(), issueBook() and returnBook(). A book that is already issued cannot be issued again, and a book that is not issued cannot be returned.

```
public class LibraryBook {
	private String title;
	private boolean issued;
	public String getTitle() {
		return this.title;
	}
	public boolean isIssued() {
		return this.issued;
	}
	public void issueBook() {
		if(this.issued==false) issued = true;
		else System.out.println("Book is already issued.");
	}
	public void returnBook() {
		if(this.issued) this.issued = false;
		else System.out.println("Book wasnt issued. book cant be returned");
	}
}
```

Problem 5

Create a class Temperature that stores the value privately in Celsius. Provide getCelsius(), getFahrenheit() and setFahrenheit(double). Reject values below absolute zero (-273.15 C). Then change the internal storage to Kelvin without changing any public method, and show that the main class still works unchanged.




```
public class Temperature {
	private double celsius;
	public Temperature(double celsius){
		if(celsius>= -273.15) this.celsius =
celsius;
		else System.out.println("Cant set below
absolute zero!");
	}
	public double getCelsius() {
		return this.celsius;
	}
	public double getFahrenheit() {
		return (celsius*9/5) + 32;
	}
	public void setFahrenheit(double f) {
		double c = (f-32)*5/9;
		if(c>= -273.15) this.celsius = c;
		else System.out.println("Cant set below absolute zero!");
	}

}
```

Class after changing the internal storage to Kelvin

```
public class Temperature {
	private double kelvin;
	public Temperature(double celsius){
		if(celsius>= -273.15) this.kelvin = celsius + 273.15;
		else System.out.println("Cant set below absolute zero!");
	}
	public double getCelsius() {
		return this.kelvin - 273.15;
	}
	public double getFahrenheit() {
		return (getCelsius()*9/5) + 32;
	}
	public void setFahrenheit(double f) {
		double c = (f-32)*5/9;
		if(c>= -273.15) this.kelvin = c + 273.15;
		else System.out.println("Cant set below absolute zero!");
	}

}
```

Main class

```
public class Main {
	public static void main(String[] args) {
	Temperature t = new Temperature(0);
	System.out.println("Celsius: " + t.getCelsius());
	System.out.println("Fahrenheit: " + t.getFahrenheit());

	t.setFahrenheit(212);

	System.out.println("\nAfter setting 212F:");
	System.out.println("Celsius: " + t.getCelsius());
	System.out.println("Fahrenheit: " + t.getFahrenheit());
	}
}
```

In both time the output is
Celsius: 0.0
Fahrenheit: 32.0
After setting 212F:
Celsius: 100.0
Fahrenheit: 212.0

Even though the internal storage is changed to Kelvin the getter and setter method are changed accordingly to work with kelvin. Thus the output remains same without changing the code of main class.

Problem 6

Create two packages, bank and app. In bank, create a class Account with one member for each modifier: private, default (package-private), protected and public. In the app, write a normal class and a subclass of Account, and try to access each member. Make a table of which access works from where, and explain why the compiler rejects some lines.

```java
package bank;
public class Account {
	private int privateVar = 10;
	int defaultVar = 20;
	protected int protectedVar = 30;
	public int publicVar = 40;
}
```

```java
//normal class
package app;
import bank.Account;
public class NormalClass {
	public void display() {
		Account a = new Account();
		a.publicVar = 100; //OK

		a.privateVar = 100; //error
		a.defaultVar  = 100; //error
		a.protectedVar = 100; //error
	}
}
```

```java
//subclass
package app;
import bank.Account;
public class SubClass extends Account{
	public void display() {
		this.publicVar = 100; //OK
		this.privateVar = 100; //error
		this.defaultVar  = 100; //error
		this.protectedVar = 100; //OK
	}
}
```

Modifier table

Modifier	Same Class	Same Package	Subclass (Different Package)	Normal Class (Different Package)
private	✅	❌	❌	❌
default (package-private)	✅	✅	❌	❌
protected	✅	✅	✅	❌
public	✅	✅	✅	✅

Some line gives error because private members are accessible only inside the class where they are declared. So the other class or sub class coundn,t access privater variable. Default variable are only

acceable within same package. Protected variable are accessible in its class and any subclass (different package included). Public variable are accessible from everywhere.

Problem 7

Design a RestaurantOrder class that manages a single customer's order for up to 10 different food items. The class must encapsulate four private fields: customerName (String), itemNames (String array), itemPrices (double array), and itemCount (int).

Implement a constructor that initializes the customer's name, along with the following public behaviors:

- addItem(String name, double price): Validates that the name is not empty, the price is greater than zero, and the 10-item capacity limit has not been exceeded before adding the item.
- calculateTotal(): Computes and returns the combined price of all items currently in the order.
- removeItem(String name): Locates and removes the specified item by name, shifting all subsequent array elements to maintain a contiguous list.
- displayOrder(): Prints a formatted summary showcasing the customer's name, each ordered item alongside its price, and the final calculated total bill.

```
class RestaurantOrder {
	private String customerName;
	private String[] itemNames;
	private double[] itemPrices;
	private int itemCount;

	public RestaurantOrder(String customerName) {
		this.customerName = customerName;
		this.itemNames = new String[10];
		this.itemPrices = new double[10];
		this.itemCount = 0;
	}

	public void addItem(String name, double price) {
		if (name == null || name.isEmpty()) {
			System.out.println("Invalid item name!");
		}
		else if (price <= 0) {
			System.out.println("Price must be greater than 0!");
		}
		else if (itemCount >= 10) {
			System.out.println("Order limit reached (10 items)!");
		}
		else {
			itemNames[itemCount] = name;
			itemPrices[itemCount] = price;
			itemCount++;
			System.out.println(name + " added successfully.");
		}
	}
	public double calculateTotal() {
		double total = 0;
		for (int i = 0; i < itemCount; i++) {
```

```
                    total += itemPrices[i];
                }
                return total;
        }
        public void removeItem(String name) {
                int index = -1;
                for (int i = 0; i < itemCount; i++) {
                        if (itemNames[i].equalsIgnoreCase(name)) {
                                index = i;
                                break;
                        }
                }
                if (index == -1) {
                System.out.println("Item not found!");
                }
                else {
                        for (int i = index; i < itemCount - 1; i++) {
                                itemNames[i] = itemNames[i + 1];
                                itemPrices[i] = itemPrices[i + 1];
                        }
                        itemNames[itemCount - 1] = null;
                        itemPrices[itemCount - 1] = 0.0;
                        itemCount--;

                }
        }
        public void displayOrder() {
                System.out.println("ORDER SUMMARY");
                System.out.println("Customer Name: " + customerName);
                if (itemCount == 0) {
                        System.out.println("No items in the order.");
                }
                else {
                        System.out.println("\nItems Ordered:");
                        for (int i = 0; i < itemCount; i++) {
                                System.out.println(itemNames[i] + " - " +itemPrices[i]);
                        }
                        System.out.println("Total Bill: " + calculateTotal());
                }
        }

}
```

Problem 8

Design a WatchList class for a streaming application that allows a user to track up to 20 unique movies. The class must strictly encapsulate two private fields: movies (String array) and movieCount (int), ensuring outside code cannot directly alter the collection state.

Implement the class with the following core operations:

- addMovie(String movie): Restricts additions to non-empty names, prevents duplicate entries, and enforces the maximum 20-movie storage limit.
- containsMovie(String movie): Evaluates the list and returns true if the movie is present, otherwise returning false.

- removeMovie(String movie): Deletes the specified movie from the tracker and shifts the remaining array elements to fill the empty slot.
- displayMovies(): Outputs an ordered, numbered list of all movies currently saved in the watchlist.
- getMovieCount(): A getter method that safely returns the current total number of movies stored in the list.

```
class WatchList {
  private String[] movies;
  private int movieCount;
  public WatchList() {
     movies = new String[20];
     movieCount = 0;
  }
  public void addMovie(String movie) {
     if (movie == null || movie.isEmpty()) {
        System.out.println("Invalid movie name.");
     }
     else if (containsMovie(movie)) {
        System.out.println("Movie already exists.");
     }
     else if (movieCount >= 20) {
        System.out.println("Watchlist is full.");
     }
     else {
           movies[movieCount] = movie;
           movieCount++;
           System.out.println(movie + " added successfully.");
     }

  }
  public boolean containsMovie(String movie) {
     for (int i = 0; i < movieCount; i++) {
        if (movies[i].equalsIgnoreCase(movie)) {
           return true;
        }
     }
     return false;
  }
  public void removeMovie(String movie) {
     int index = -1;
     for (int i = 0; i < movieCount; i++) {
        if (movies[i].equalsIgnoreCase(movie)) {
           index = i;
           break;
        }
     }
     if (index == -1) {
        System.out.println("Movie not found.");
     }
     else {
           for (int i = index; i < movieCount - 1; i++) {
           movies[i] = movies[i + 1];
        }
        movies[movieCount - 1] = null;
        movieCount--;
        System.out.println(movie + " removed successfully.");
     }
```

```
    }
    public void displayMovies() {
        if (movieCount == 0) {
            System.out.println("Watchlist is empty.");
        }
        else {
                System.out.println("WATCHLIST");
                    for (int i = 0; i < movieCount; i++) {
                        System.out.println((i + 1) + ". " + movies[i]);
                    }
        }

    }
    public int getMovieCount() {
        return this.movieCount;
    }
}
class WatchList {
    private String[] movies;
    private int movieCount;
    public WatchList() {
        movies = new String[20];
        movieCount = 0;
    }
    public void addMovie(String movie) {
        if (movie == null || movie.isEmpty()) {
            System.out.println("Invalid movie name.");
        }
        else if (containsMovie(movie)) {
            System.out.println("Movie already exists.");
        }
        else if (movieCount >= 20) {
            System.out.println("Watchlist is full.");
        }
        else {
                movies[movieCount] = movie;
                movieCount++;
                System.out.println(movie + " added successfully.");
        }

    }
    public boolean containsMovie(String movie) {
        for (int i = 0; i < movieCount; i++) {
            if (movies[i].equalsIgnoreCase(movie)) {
                return true;
            }
        }
        return false;
    }
    public void removeMovie(String movie) {
        int index = -1;
        for (int i = 0; i < movieCount; i++) {
            if (movies[i].equalsIgnoreCase(movie)) {
                index = i;
                break;
            }
        }
        if (index == -1) {
```

```
            System.out.println("Movie not found.");
        }
        else {
                for (int i = index; i < movieCount - 1; i++) {
                 movies[i] = movies[i + 1];
            }
            movies[movieCount - 1] = null;
            movieCount--;
            System.out.println(movie + " removed successfully.");
        }

    }
    public void displayMovies() {
        if (movieCount == 0) {
            System.out.println("Watchlist is empty.");
        }
        else {
                System.out.println("WATCHLIST");
                    for (int i = 0; i < movieCount; i++) {
                        System.out.println((i + 1) + ". " + movies[i]);
                    }
        }

    }
    public int getMovieCount() {
        return this.movieCount;
    }
}
```